ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Departamento de Ciencias de la Computación
Ingeniería en Software

Aplicaciones Distribuidas

Unidad 1

FUNDAMENTOS DE APLICACIONES DISTRIBUIDAS

Profesor: Geovanny Cudco

Sílabo

Unidad I: FUNDAMENTOS DE APLICACIONES DISTRIBUIDAS

- **Introducción**
  - Conceptos de Aplicaciones Distribuidas
  - Evolución de las aplicaciones distribuidas
  - Principales desafíos y ventajas
  - Falacias de las aplicaciones Distribuidas
  - Teorema CAP
- **Gestión de concurrencia**
  - Concurrencia en aplicaciones distribuidas
  - Hilos de ejecución en la programación distribuida
- **Sockets**
  - Introducción
  - Comunicación en aplicaciones distribuidas
  - Protocolos de la Capa de Aplicación
- **Fundamentos de WebSockets**
  - Conceptos y definiciones
  - Desarrollo e implementación de WebSockets
  - Comunicación en Tiempo Real

Unidad II: TECNOLOGÍAS PARA EL DESARROLLO DE APLICACIONES DISTRIBUIDAS

- Implementación de servicios
  - Principios SOLID
  - Patrones de Diseño
  - Manejo de Transacciones ACID
  - Manejo de Excepciones en capa de Servicios
- Observabilidad
  - Definición y Conceptos Teóricos
  - Implementación de Logs
- Autenticación basada en tokens
  - OAuth 2.0
  - JWT
  - Servicios de Identidad

Unidad III: ARQUITECTURAS DE APLICACIONES DISTRIBUIDAS

- Patrones de Comunicación Síncrona y Asíncrona
  - Fundamentos de RPC
  - Colas de Mensajes
  - Servidores de Colas
  - Publicación y consumo de mensajes
- Arquitecturas de Alta Disponibilidad y Escalabilidad
  - Introducción a arquitecturas cluster
  - Nodos
  - Redes de interconexión
- Arquitecturas Cluster
  - Software y herramientas
  - Aplicaciones Clusters
    - Alta productividad
    - Alto rendimiento
    - Alta Disponibilidad
  - Balanceo de Carga

TÉCNICAS Y PONDERACION DE LA EVALUACIÓN

Técnica de evaluación	1er Parcial	2do Parcial	3er Parcial
Tareas o guías/Pruebas oral/escrita	4	4	4
Laboratorios/Informes	4	4	4
Proyectos	5	5	5
Examen Parcial	7	7	7
TOTAL	**20**	**20**	**20**

ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Unidad I: FUNDAMENTOS DE APLICACIONES DISTRIBUIDAS

Introducción

Introducción a las Aplicaciones Distribuidas

1 Introducción

1.1 Conceptos de Aplicaciones Distribuidas

1.2 Evolución de las aplicaciones distribuidas

1.3 Principales desafíos y ventajas

1.4 Falacias de las aplicaciones Distribuidas

1.5 Teorema CAP

1.1 Conceptos de Aplicaciones Distribuidas

Sistema

Aplicación

1.1 Conceptos de Aplicaciones Distribuidas

Sistema

- Conjunto de elementos interrelacionados que funcionan como un todo organizado para alcanzar un objetivo común. Puede ser físico, biológico, social o informático. (ej. el cuerpo humano, una empresa, un ecosistema, un sistema informático.)
- **Conjunto de elementos** (hardware, software, procesos, personas) **que interactúan entre sí para cumplir objetivos comunes.**

Aplicación

- **Programa** o conjunto de **programas diseñados para realizar tareas específicas que satisfacen necesidades concretas** de usuarios o de negocio.

1.1 Conceptos de Aplicaciones Distribuidas

Sistema

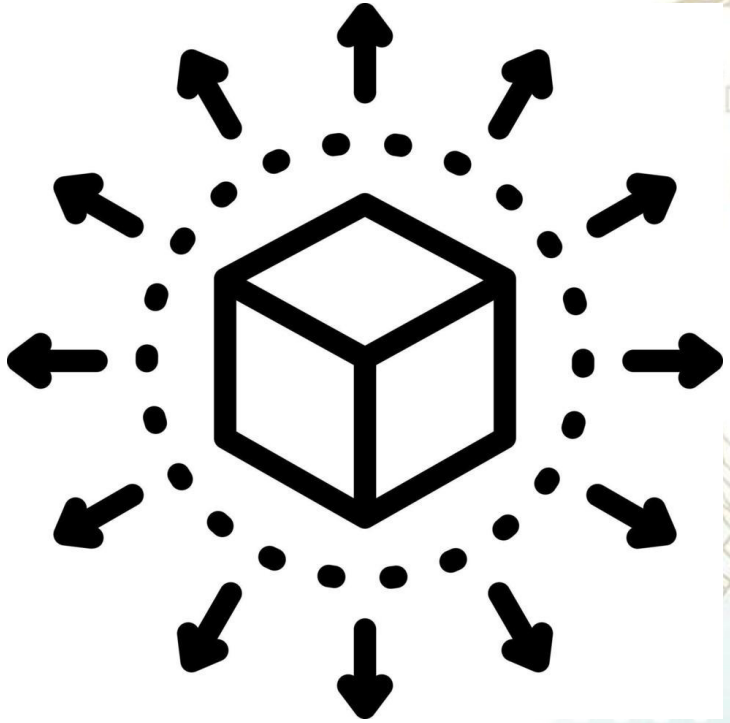
- Incluye el **hardware** (servidores, ordenadores, dispositivos de red), el **sistema operativo** (Linux, Windows), y el **middleware** (servidores de aplicaciones, gestores de mensajería) que proveen las bases sobre las que se ejecuta el software.
- **Gestiona recursos** (CPU, memoria, disco, red), ofrece servicios de bajo nivel (comunicación, seguridad, persistencia), y garantiza propiedades como tolerancia a fallos, escalabilidad o consistencia de datos.

Aplicación

- Se apoya en el sistema (hardware, OS y middleware) para ejecutar la **lógica de negocio**, presentar interfaces al usuario y acceder a datos.
- Suele componerse de módulos o servicios (backend, frontend, base de datos) que colaboran entre sí.

1.1 Conceptos de Aplicaciones Distribuidas

¿Qué es Distribución?

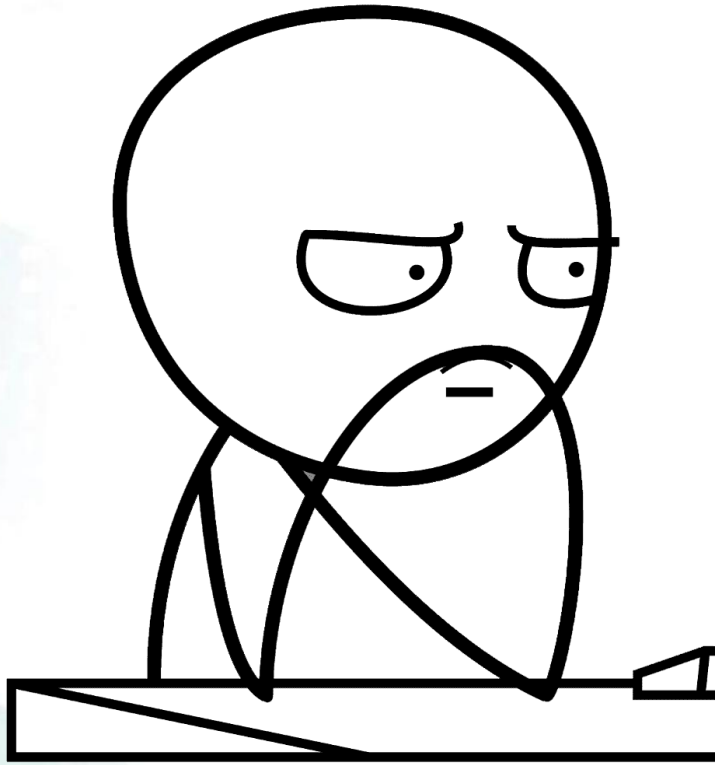


1.1 Conceptos de Aplicaciones Distribuidas

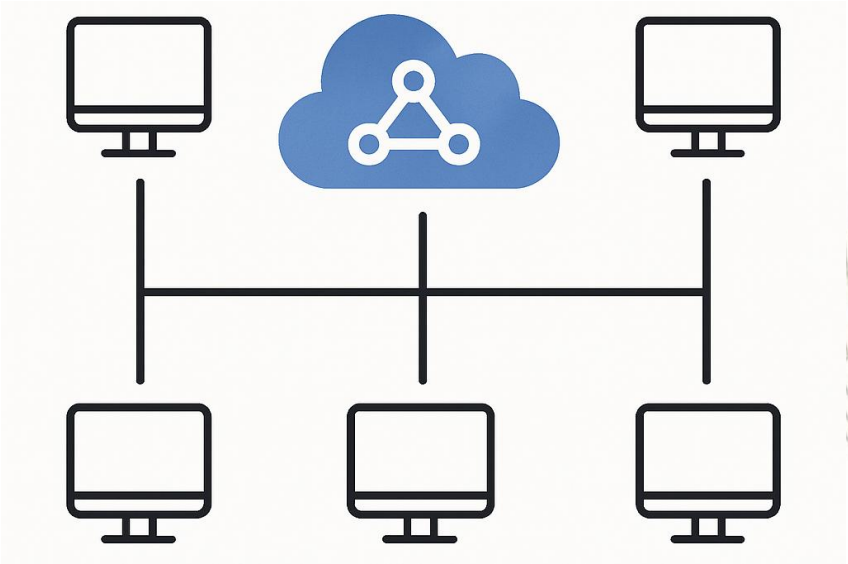
¿Qué es Distribución?

- Técnica de **dividir componentes** de un sistema o aplicación **en varias ubicaciones físicas o lógicas**.
- Estos componentes pueden estar en diferentes máquinas conectadas en red, trabajando juntas para lograr un objetivo común.

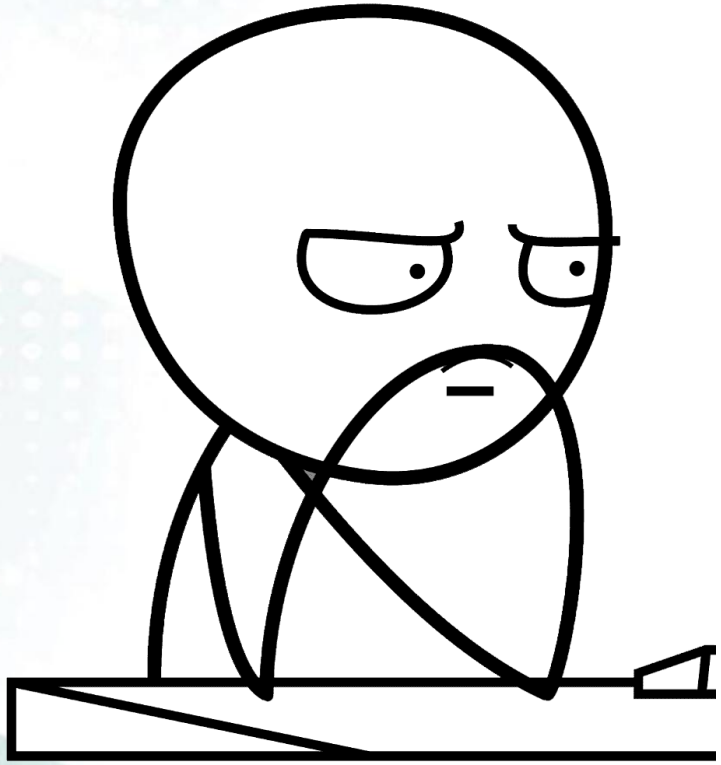
1.1 Conceptos de Aplicaciones Distribuidas



Sistemas distribuidos



1.1 Conceptos de Aplicaciones Distribuidas

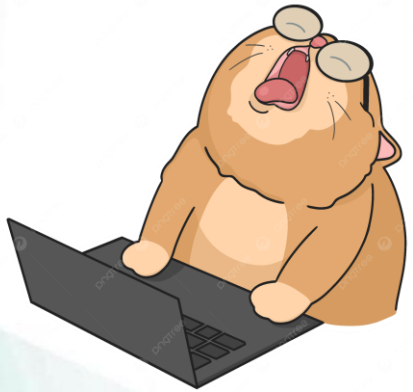


Sistemas distribuidos

- **Conjunto de computadoras independientes** que se presentan a los usuarios **como un sistema único y coherente.**
- Estas computadoras **colaboran mediante una red para compartir recursos, procesamiento y almacenamiento.**

1.1 Conceptos de Aplicaciones Distribuidas

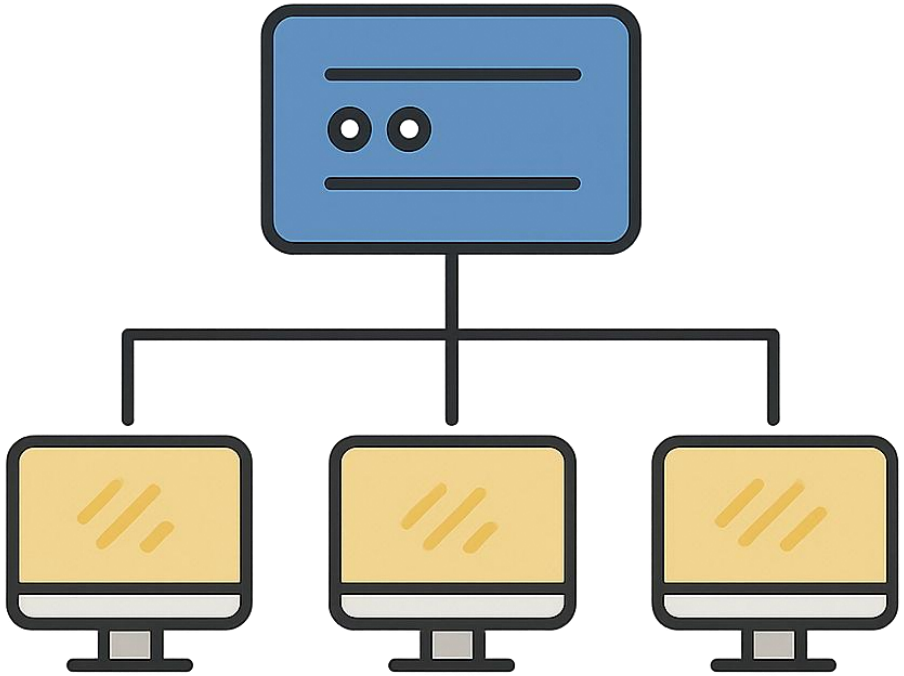
«Un sistema distribuido es una colección de computadoras independientes que aparecen ante los usuarios del sistema como una única computadora» Tanenbaum.



«Sistemas Distribuidos son aquellos en los cuales los componentes de hardware y software están ubicados en computadoras de una red y se comunican y coordinan sus acciones solamente por medio de mensajes» Coulouris.

1.1 Conceptos de Aplicaciones Distribuidas

Aplicación Centralizada

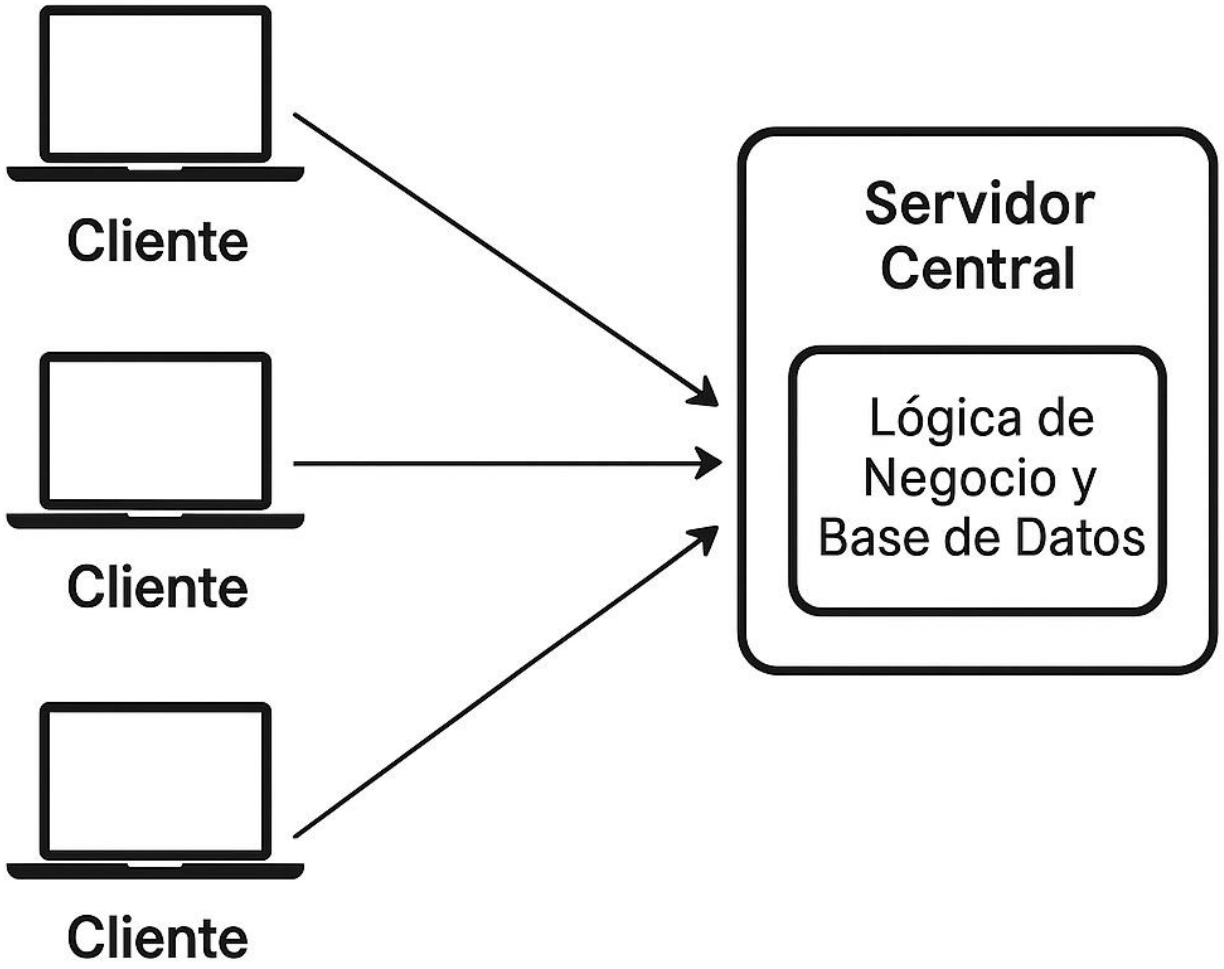


1.1 Conceptos de Aplicaciones Distribuidas

Aplicación Centralizada

Es aquella en la que todos los procesos, datos y control se gestionan desde un único servidor central. Los usuarios acceden a esta aplicación a través de terminales o dispositivos cliente, pero toda la lógica de negocio y almacenamiento se encuentra en un solo lugar.

1.1 Conceptos de Aplicaciones Distribuidas



1.1 Conceptos de Aplicaciones Distribuidas

Aplicación Centralizada

Sistemas de Gestión Académica Local (ej. SAE - Sistema Académico Escolar)

Explicación: Toda la información sobre estudiantes, docentes, calificaciones y asistencia se almacena y gestiona desde un único servidor en la institución educativa. Los usuarios (docentes y administrativos) acceden desde sus computadoras, **pero si el servidor central falla, no pueden acceder al sistema.**

1.1 Conceptos de Aplicaciones Distribuidas

Aplicación Centralizada

Aplicaciones de Punto de Venta (POS) Locales

Explicación: En pequeños comercios, los sistemas de ventas y facturación están conectados a un servidor en la tienda, donde se almacena la información de productos y ventas. **No funcionan fuera de ese entorno.**

1.1 Conceptos de Aplicaciones Distribuidas

En resumen

- Todos los clientes acceden a un único **servidor.**
- Toda la lógica de negocio y base de datos está en ese servidor.
- La caída del servidor implica pérdida de servicio para todos.

1.1 Conceptos de Aplicaciones Distribuidas

Las **APLICACIONES DISTRIBUIDAS** son sistemas cuyos componentes de software se ejecutan en múltiples máquinas (nodos) interconectadas por una red, cooperando entre sí para ofrecer una funcionalidad unificada al usuario final.

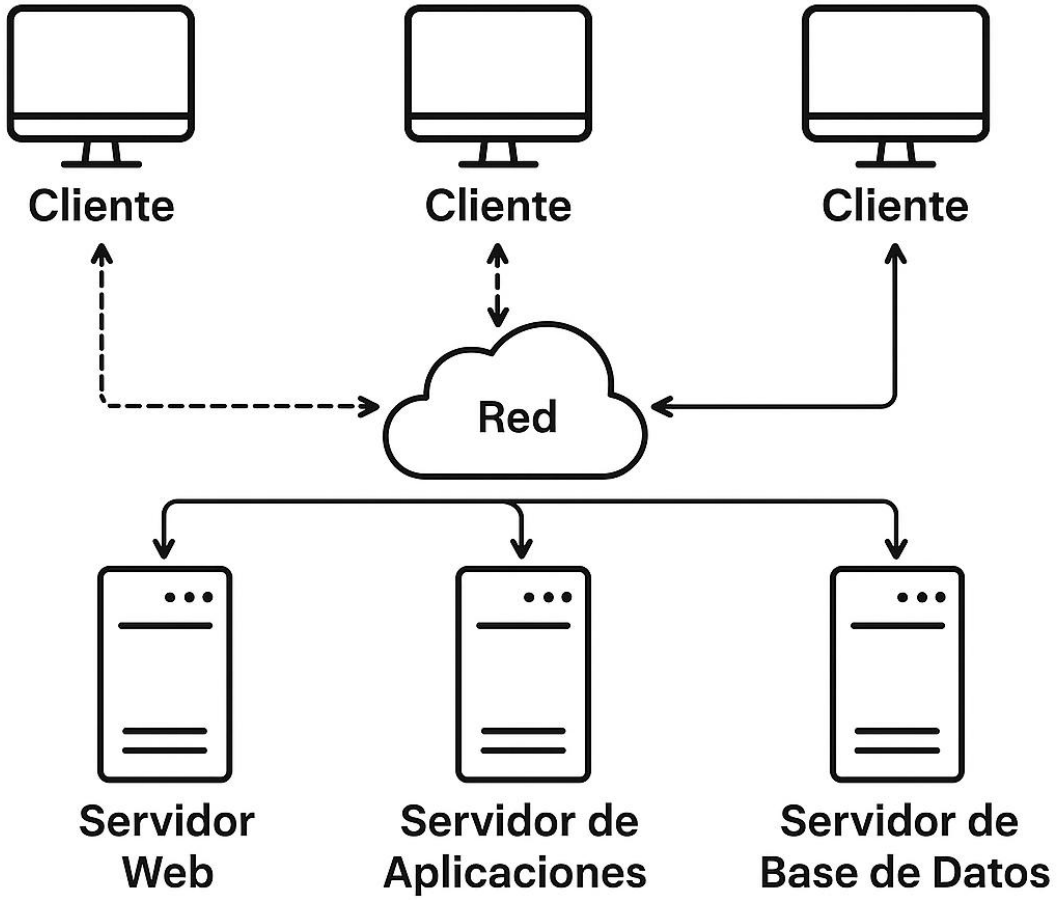
Es una **aplicación** con **distintos componentes** que se ejecutan **en entornos separados**, normalmente en diferentes plataformas conectadas a través de una red.

1.1 Conceptos de Aplicaciones Distribuidas

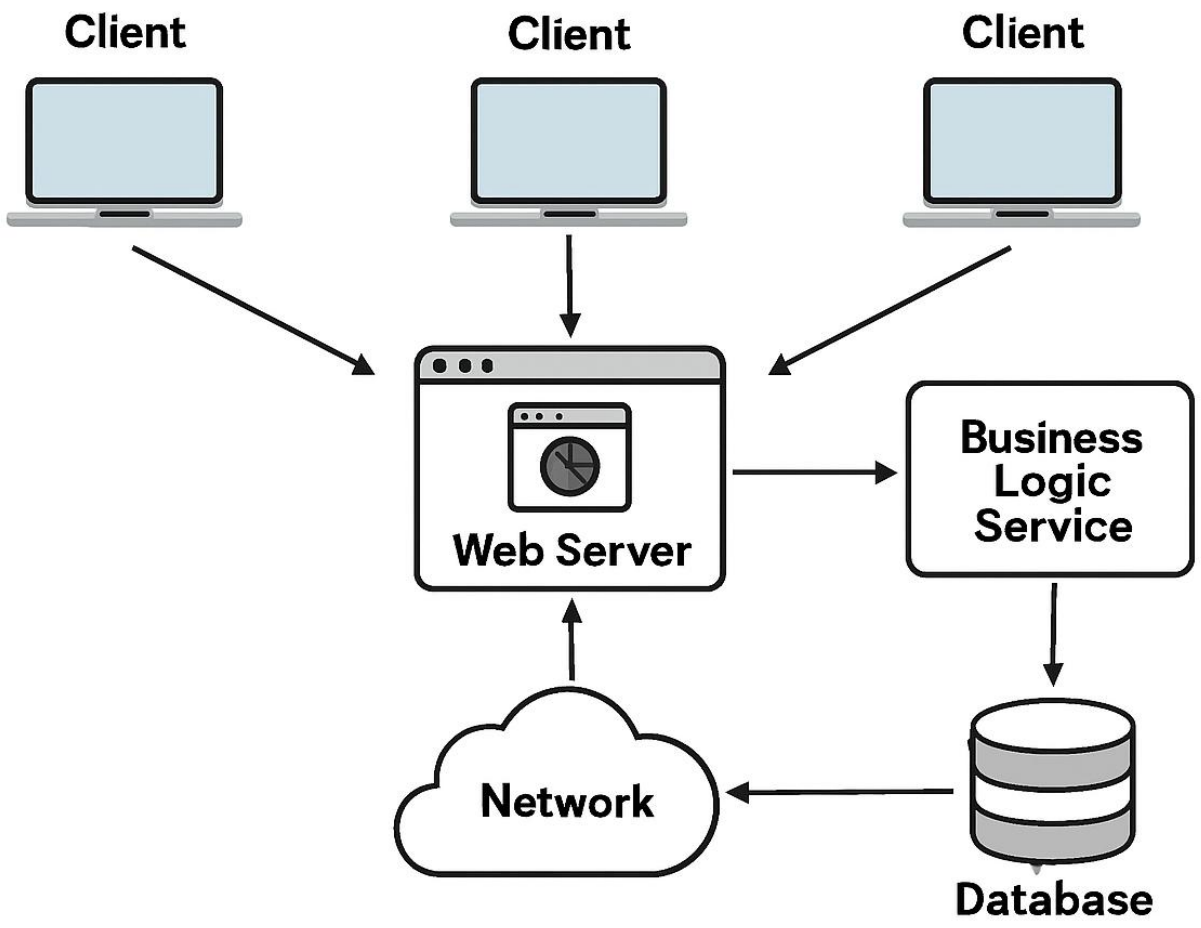
Las **APLICACIONES DISTRIBUIDAS** son sistemas cuyos componentes de software se ejecutan en múltiples máquinas (nodos) interconectadas por una red, cooperando entre sí para ofrecer una funcionalidad unificada al usuario final.

Es una aplicación con distintos componentes que se ejecutan en entornos separados, normalmente en diferentes plataformas conectadas a través de una red.

1.1 Conceptos de Aplicaciones Distribuidas



1.1 Conceptos de Aplicaciones Distribuidas



Laboratorio 1: Implementación de un Servidor DNS y Resolución de Nombre de Dominio en entorno virtualizado

1. Objetivo

Diseñar y desplegar un entorno de red práctico que permita comprender el funcionamiento de un servidor DNS, configurarlo en una máquina virtual, apuntar registros a un servidor web en otra VM y verificar la resolución de dominio desde un cliente en una tercera VM.

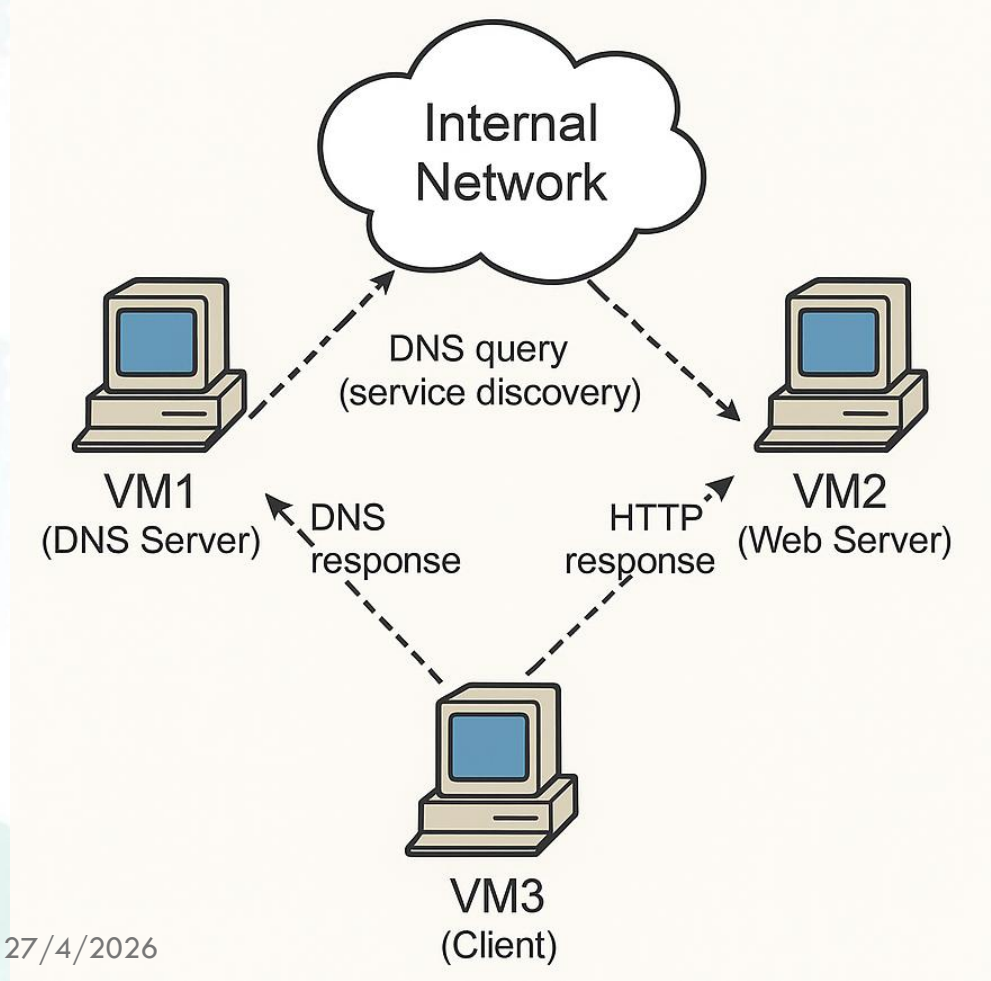
Laboratorio 1: Implementación de un Servidor DNS y Resolución de Nombre de Dominio en entorno virtualizado

2. Descripción

En este taller los estudiantes crearán tres máquinas virtuales (VM1, VM2 y VM3) interconectadas en una red privada:

- VM1 (DNS Server): Instalación y configuración de un servicio DNS (por ejemplo BIND9) para gestionar la zona tudominio.local.
- VM2 (Web Server): Despliegue de un servidor web (por ejemplo Apache o Nginx) que sirva una página de prueba en HTTP.
- VM3 (Cliente DNS): Configuración para que resuelva el nombre www.tudominio.local mediante la VM1 y acceda al contenido servido por VM2.

Laboratorio 1: Implementación de un Servidor DNS y Resolución de Nombre de Dominio en entorno virtualizado



VM1 (DNS Server): Atiende consultas DNS (tanto de VM3 como de VM2 para service discovery).

VM2 (Web Server): Realiza una consulta DNS inicial a VM1 y atiende peticiones HTTP de VM3.

VM3 (Client):

- Envía DNS query a VM1 → recibe DNS response
- Envía HTTP request a VM2 → recibe HTTP response

Laboratorio 1: Implementación de un Servidor DNS y Resolución de Nombre de Dominio en entorno virtualizado

3. Entregables

- Demostración en vivo del funcionamiento del Sistema Distribuido.
- Informe con las siguientes secciones: Introducción, Objetivos, Metodología, Resultados obtenidos, Conclusiones y Bibliografía.

Laboratorio 1: Implementación de un Servidor DNS y Resolución de Nombre de Dominio en entorno virtualizado

¿POR QUÉ ESTE TALLER CONSTITUYE UN SISTEMA DISTRIBUIDO?

Múltiples nodos autónomos

- Cada máquina virtual (VM1, VM2 y VM3) funciona de manera independiente, con su propio sistema operativo, servicios y configuraciones.
- No existe un único "equipo central"; cada nodo aporta recursos de computación y storage.

Cooperación vía red

- VM3 (cliente) envía consultas DNS sobre red interna a VM1, que responde con datos necesarios para conectarse a VM2.
- VM2 publica su servicio web en IP interna, y VM3 lo consume usando el nombre de dominio resuelto por VM1.

Separa responsabilidades

- VM1 se encarga exclusivamente de la resolución de nombres (servicio de directorio distribuido).
- VM2 se dedica al servicio HTTP (servidor web).
- VM3 actúa como consumidor y validador, usando ambos servicios.

Laboratorio 1: Implementación de un Servidor DNS y Resolución de Nombre de Dominio en entorno virtualizado

¿POR QUÉ ESTE TALLER CONSTITUYE UN SISTEMA DISTRIBUIDO?

Escalabilidad y flexibilidad

- Se pueden añadir más servidores web (más VMs como VM2) y apuntar con nuevos registros DNS en VM1 sin cambiar la configuración del cliente.
- Si el servicio crece, basta desplegar instancias adicionales de cada rol y balancear la carga.

Transparencia de ubicación

- VM3 accede a “www.tudominio.local” sin importar la IP física de VM2; el cliente no necesita conocer la ubicación real del servidor web, sólo el nombre de dominio.

Introducción a las Aplicaciones Distribuidas

1 Introducción

1.1 Conceptos de Aplicaciones Distribuidas

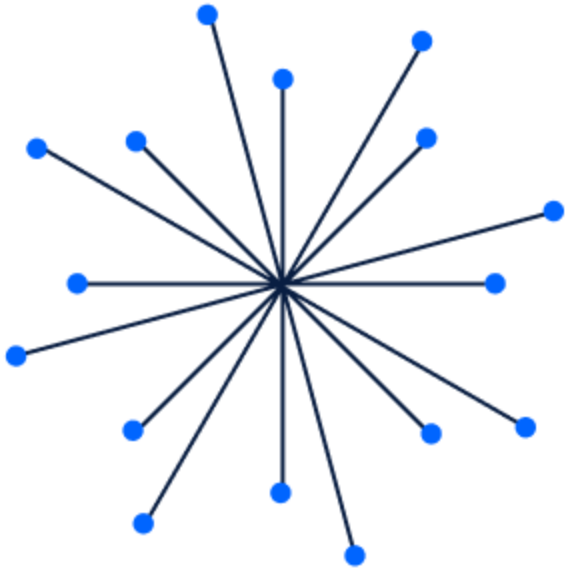
1.2 Evolución de las aplicaciones distribuidas

1.3 Principales desafíos y ventajas

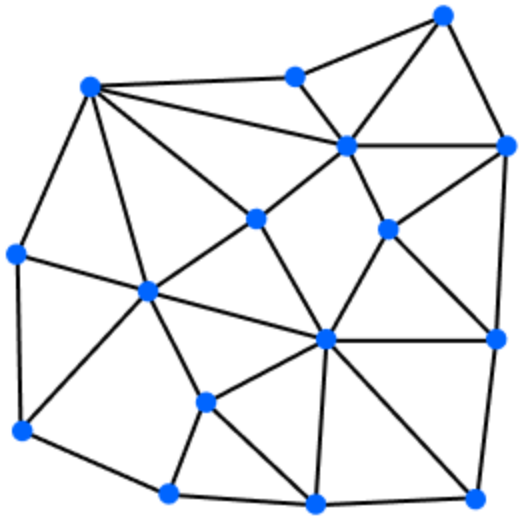
1.4 Falacias de las aplicaciones Distribuidas

1.5 Teorema CAP

Evolución de las Aplicaciones Distribuidas



Centralized



Distributed

Evolución de las Aplicaciones Distribuidas

Décadas 1960–1970: Orígenes y Primeros Sistemas

- **1960:**
  - Se desarrollan los primeros sistemas operativos multiusuario y multitarea como **CTSS** (MIT, 1961) y **UNIX** (AT&T Bell Labs, 1969).
  - Surge la necesidad de compartir recursos computacionales entre usuarios remotos.
- **1969:**
  - Se crea **ARPANET**, la precursora de Internet, lo que permite la interconexión de computadoras a gran escala.
  - Se considera el punto de partida de la **computación distribuida**.
- **1972:**
  - Se desarrolla el **correo electrónico en ARPANET**, considerada la **primera aplicación distribuida a gran escala**.

Evolución de las Aplicaciones Distribuidas

Década 1980: Redes Locales y Cliente-Servidor

- **1980:**
  - Surgen las **redes de área local (LAN)** como Ethernet, permitiendo la conexión de computadoras dentro de una misma organización.
  - Se consolida el modelo **cliente-servidor** como arquitectura estándar para aplicaciones distribuidas.
- **1982:**
  - Se celebra la primera conferencia académica especializada: **Symposium on Principles of Distributed Computing (PODC)**.
- **1985:**
  - Se inicia el proyecto **ANSA**, uno de los primeros intentos por crear una arquitectura de objetos distribuidos.
- **1989:**
  - Se funda el **Object Management Group (OMG)**, que más tarde desarrollará **CORBA**.

Evolución de las Aplicaciones Distribuidas

Década 1990: Objetos Distribuidos y Middleware

- **1990:**
  - Se publica la primera especificación de **CORBA** (Common Object Request Broker Architecture), un estándar para objetos distribuidos multiplataforma.
- **1992:**
  - Aparecen las **primeras implementaciones comerciales de CORBA**.
- **1993:**
  - Se introduce **CGI** (Common Gateway Interface), permitiendo la creación de páginas web dinámicas y la interacción entre aplicaciones web y servidores.
- **1995:**
  - Java introduce **RMI** (Remote Method Invocation), permitiendo la invocación de métodos entre objetos Java en diferentes máquinas.
- **1999:**
  - Se popularizan las **aplicaciones cliente-servidor** en entornos empresariales, con herramientas como **DCOM** (Microsoft) y **CORBA** en diversas industrias.

Evolución de las Aplicaciones Distribuidas

Década 2000: Web Services y SOA

- **2000:**
  - Roy Fielding introduce **REST** (Representational State Transfer) en su tesis doctoral, sentando las bases para los **servicios web modernos.**
- **2001–2003:**
  - Se consolida **SOAP** (Simple Object Access Protocol) como estándar para servicios web basados en XML.
- **2005:**
  - Se populariza la **Arquitectura Orientada a Servicios (SOA),** promoviendo la reutilización de servicios distribuidos.
- **2007:**
  - Surgen las primeras **plataformas de computación en la nube,** como **Google App Engine** (2008), sentando las bases para el despliegue de aplicaciones distribuidas en la nube.

Evolución de las Aplicaciones Distribuidas

Década 2010: Cloud, Microservicios y Contenedores

- **2013:**
  - Se presenta **Docker,** revolucionando la forma de empaquetar y desplegar aplicaciones distribuidas con **contenedores.**
- **2014:**
  - Amazon lanza **AWS Lambda,** marcando el inicio de la **computación serverless.**
- **2015:**
  - Se lanza **Kubernetes,** una plataforma de **orquestación de contenedores** que se convierte en estándar para aplicaciones distribuidas en la nube.
- **2016:**
  - Google y Microsoft lanzan **Google Cloud Functions** y **Azure Functions,** respectivamente, consolidando el modelo **serverless.**
- **2017:**
  - Se consolida la **arquitectura de microservicios** como enfoque dominante para aplicaciones empresariales distribuidas.

Evolución de las Aplicaciones Distribuidas

Década 2020: Nativo en la Nube y Automatización

- **2020:**
  - Se populariza el concepto de **aplicaciones nativas de la nube (cloud-native)**, combinando microservicios, contenedores, Kubernetes y DevOps.
- **2021–2025:**
  - Se adoptan ampliamente **plataformas serverless** y **arquitecturas event-driven**.
  - Se integran tecnologías como **mallas de servicio (Service Mesh)** e **infraestructura inmutable**.
  - Se promueve la **automatización total** del ciclo de vida de aplicaciones distribuidas con **GitOps, CI/CD** y **observabilidad avanzada**.

Introducción a las Aplicaciones Distribuidas

1 Introducción

1.1 Conceptos de Aplicaciones Distribuidas

1.2 Evolución de las aplicaciones distribuidas

1.3 Principales desafíos y ventajas

1.4 Falacias de las aplicaciones Distribuidas

1.5 Teorema CAP

1.3 Principales desafíos y ventajas

Ventaja	Descripción
Escalabilidad horizontal	❖ Permite añadir más nodos (servidores, contenedores, dispositivos) para manejar mayor carga, en lugar de depender de hardware más potente. ❖ Esto es clave en entornos cloud y para aplicaciones con tráfico variable (ej. e-commerce en temporadas altas).
Alta disponibilidad y tolerancia a fallos	❖ Al distribuir la carga y replicar servicios en múltiples nodos o regiones, el sistema puede seguir funcionando, aunque uno o varios componentes fallen. ❖ Esto se logra mediante redundancia y patrones como el "disyuntor" (circuit breaker).

1.3 Principales desafíos y ventajas

Ventaja	Descripción
Mejor rendimiento mediante procesamiento paralelo	Las tareas complejas pueden dividirse y ejecutarse simultáneamente en distintos nodos, acelerando su resolución (ej. análisis de big data con Spark, IA distribuida).
Compartición eficiente de recursos	Permite el uso compartido de hardware costoso (almacenamiento, GPUs, impresoras) y datos entre múltiples usuarios o sistemas, optimizando costos y acceso.
Flexibilidad tecnológica	Cada componente (microservicio, nodo) puede usar la tecnología más adecuada para su función (lenguaje, base de datos, framework), facilitando la innovación y el mantenimiento.

1.3 Principales desafíos y ventajas

Desventaja	Descripción
Mayor complejidad de desarrollo y mantenimiento	❖ Coordinar múltiples servicios, gestionar la comunicación asíncrona, depurar fallos distribuidos y manejar versiones compatibles requiere herramientas y conocimientos avanzados.
Costos operativos más altos	❖ Aunque el hardware puede ser más económico, los costos de red, monitoreo, orquestación (Kubernetes), seguridad, y personal especializado suelen ser significativos.
Sobrecarga de red y procesamiento	❖ La comunicación constante entre nodos consume ancho de banda, CPU y memoria. ❖ Protocolos, serialización, autenticación y reintentos añaden latencia y consumo innecesario si no se optimizan.

1.3 Principales desafíos y ventajas

Desventaja	Descripción
Superficie de ataque ampliada	❖ Cada nodo, API o conexión es un posible punto de entrada para atacantes. ❖ La seguridad debe implementarse en múltiples capas (red, aplicación, datos), lo que es más difícil que en sistemas centralizados.
Dificultad para garantizar la coherencia fuerte	❖ Mantener todos los datos sincronizados en tiempo real entre nodos es costoso y a menudo inviable. ❖ Muchos sistemas optan por coherencia eventual, lo que puede generar inconsistencias temporales.

1.3 Principales desafíos y ventajas

Desafíos	Descripción
Gestión de la latencia de red	❖ La comunicación entre nodos introduce retrasos impredecibles. ❖ Esto afecta la experiencia del usuario y la lógica de negocio, especialmente en llamadas encadenadas (problema del "chatty API").
Observabilidad y monitoreo integral	❖ Es difícil rastrear una transacción que atraviesa decenas de servicios. ❖ Se requieren herramientas avanzadas de logging, métricas y trazado distribuido (como OpenTelemetry) para diagnosticar problemas.

1.3 Principales desafíos y ventajas

Desafíos	Descripción
Coordinación de transacciones distribuidas	❖ Las transacciones ACID tradicionales no se aplican fácilmente. ❖ Se deben usar patrones alternativos como Saga, transacciones compensatorias o CQRS, que son más complejos de implementar.
Gestión de la heterogeneidad	❖ Integrar dispositivos, protocolos, lenguajes y sistemas operativos diversos (especialmente en IoT o edge) requiere estándares abiertos y middleware robusto para garantizar interoperabilidad.
Actualización y despliegue continuo seguro	❖ Desplegar cambios en cientos de microservicios sin interrumpir el servicio exige estrategias como canary releases, blue-green deployments y pruebas automatizadas exhaustivas.

Introducción a las Aplicaciones Distribuidas

1 Introducción

1.1 Conceptos de Aplicaciones Distribuidas

1.2 Evolución de las aplicaciones distribuidas

1.3 Principales desafíos y ventajas

1.4 Falacias de las aplicaciones Distribuidas

1.5 Teorema CAP

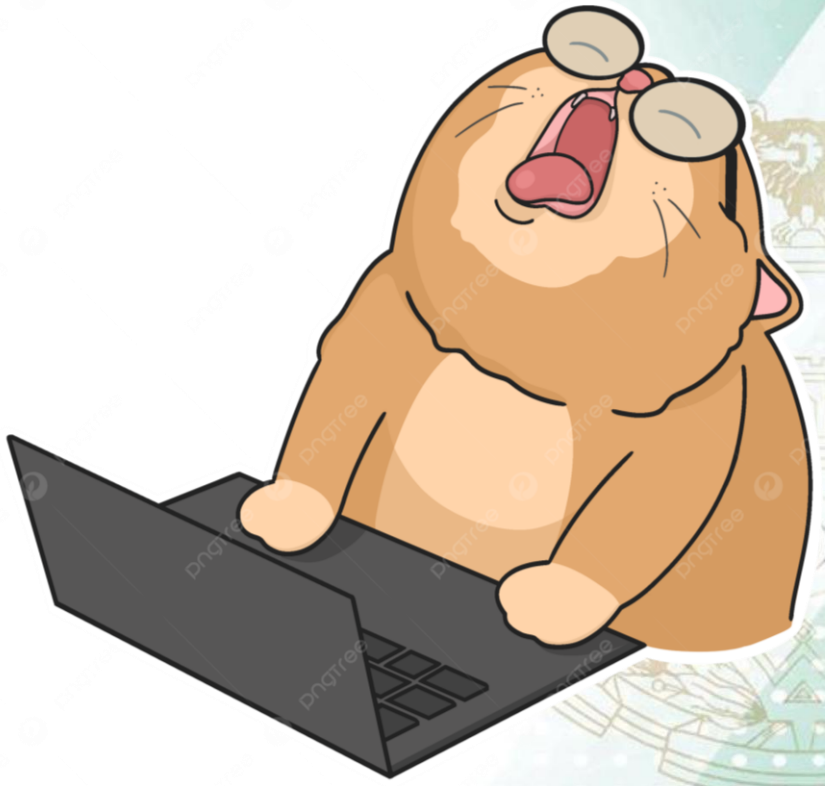
1.4 Falacias de las aplicaciones Distribuidas

- Son una lista clásica de suposiciones erróneas que los desarrolladores suelen hacer al diseñar aplicaciones distribuidas.
- Fueron formuladas originalmente por L. Peter Deutsch y otros ingenieros de Sun Microsystems en la década de 1990.
- Siguen siendo profundamente relevantes hoy en día, especialmente con la proliferación de microservicios, nube, IoT y edge computing.

1.4 Falacias de las aplicaciones Distribuidas

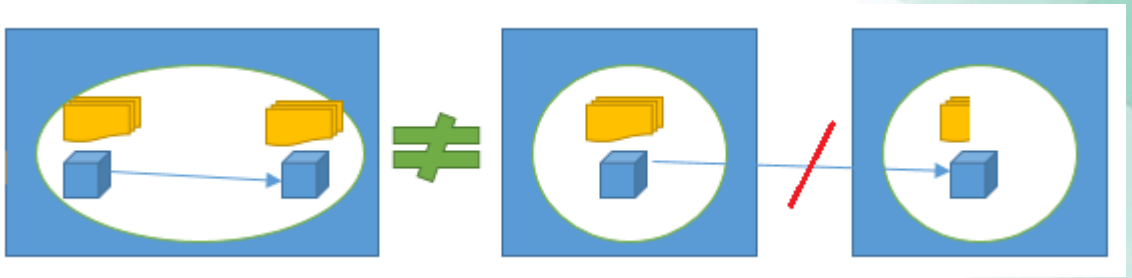
Falacias

- La red es fiable
- La latencia es cero
- El ancho de banda es infinito
- La red es segura
- La topología de la red no cambia
- Hay un solo administrador de la red
- El costo del transporte es cero
- La red es homogénea



1.4 Falacias de las aplicaciones Distribuidas

LA RED ES FIABLE



¿Por qué es una falacia?

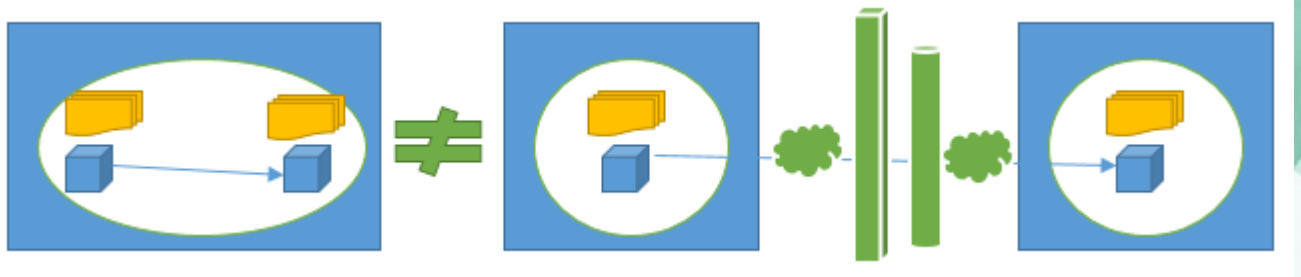
- Las redes fallan constantemente: cortes de fibra, errores de hardware, congestión, problemas de ISP, etc.
- La red nunca es 100% confiable.

Consecuencias prácticas si se cree verdadera

- Aplicaciones que no manejan reintentos, timeouts o caídas de red se bloquean o pierden datos.
- Ejemplo: una llamada a un microservicio sin manejo de errores puede colapsar todo el flujo.

1.4 Falacias de las aplicaciones Distribuidas

LA LATENCIA ES CERO



¿Por qué es una falacia?

- La comunicación a través de la red siempre tiene retraso (latencia), que varía según la distancia, congestión y protocolo.
- Incluso en la misma región cloud, puede haber decenas de milisegundos.

Consecuencias prácticas si se cree verdadera

- Diseños que asumen respuestas instantáneas (ej. llamadas síncronas encadenadas) generan interfaces lentas, timeouts y mala experiencia de usuario.

1.4 Falacias de las aplicaciones Distribuidas

EL ANCHO DE BANDA ES INFINITO

¿Por qué es una falacia?

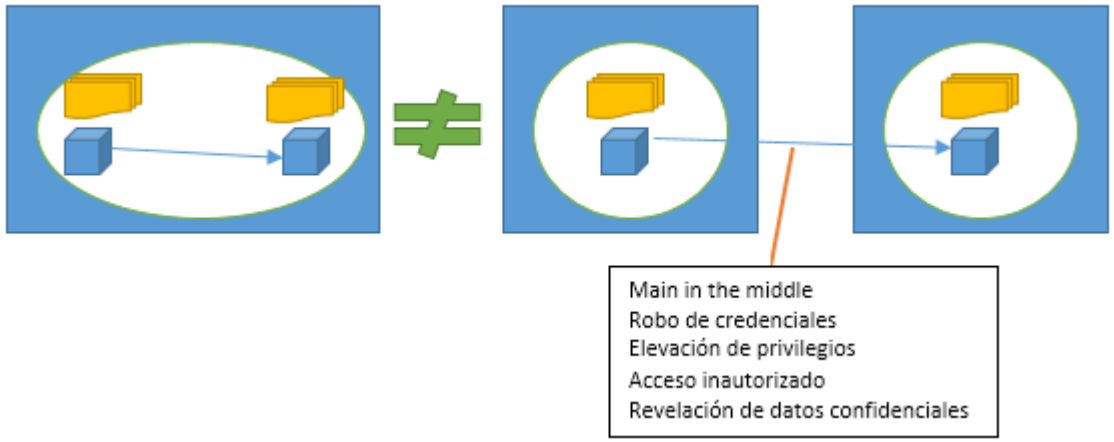
- El ancho de banda es limitado y compartido.
- En entornos móviles, IoT o redes congestionadas, puede ser muy bajo..

Consecuencias prácticas si se cree verdadera

- Transferir grandes volúmenes de datos sin compresión o paginación satura la red, ralentiza otros servicios y aumenta costos (especialmente en la nube).

1.4 Falacias de las aplicaciones Distribuidas

LA RED ES SEGURA



¿Por qué es una falacia?

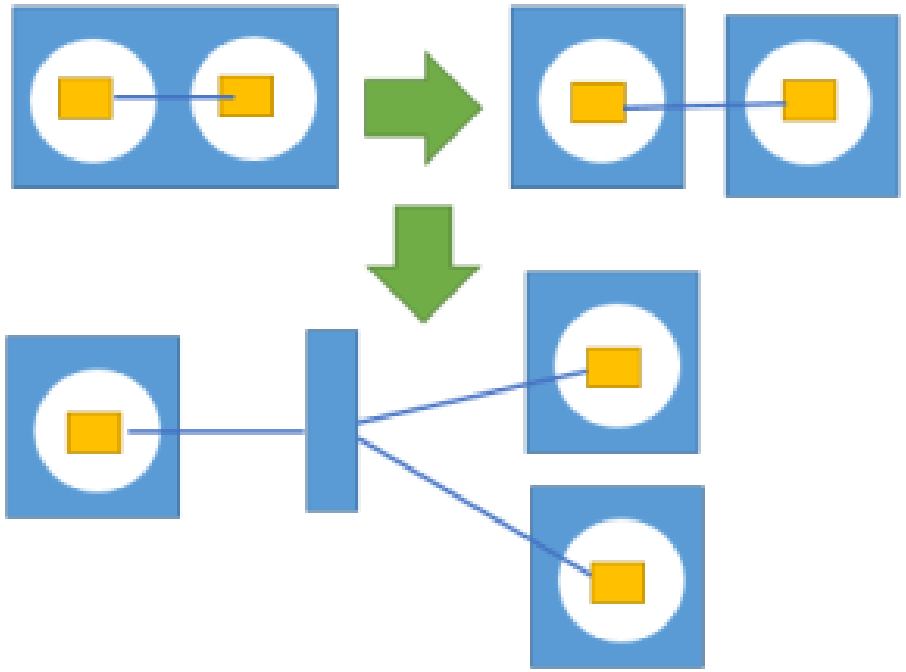
- Las redes, especialmente las públicas o inalámbricas, están expuestas a sniffing, MITM (hombre en el medio), DDoS, inyecciones, etc..

Consecuencias prácticas si se cree verdadera

- Transmitir datos sin cifrado (TLS), usar credenciales débiles o confiar ciegamente en la red expone información sensible y permite ataques.

1.4 Falacias de las aplicaciones Distribuidas

LA TOPOLOGÍA DE LA RED NO CAMBIA



¿Por qué es una falacia?

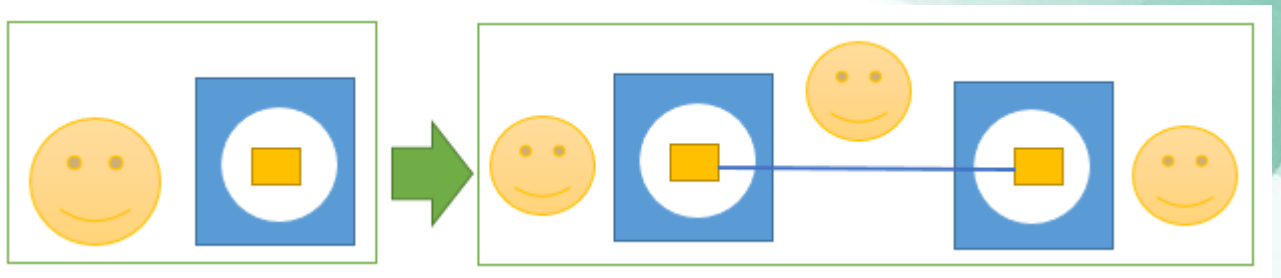
- Los nodos se mueven (dispositivos móviles), se agregan o eliminan (escalado automático), y las rutas de red cambian constantemente (balanceo de carga, fallos).

Consecuencias prácticas si se cree verdadera

- Aplicaciones que almacenan direcciones IP fijas o asumen rutas estáticas fallan al migrar a la nube, usar contenedores o implementar edge computing.

1.4 Falacias de las aplicaciones Distribuidas

HAY UN SOLO ADMINISTRADOR



¿Por qué es una falacia?

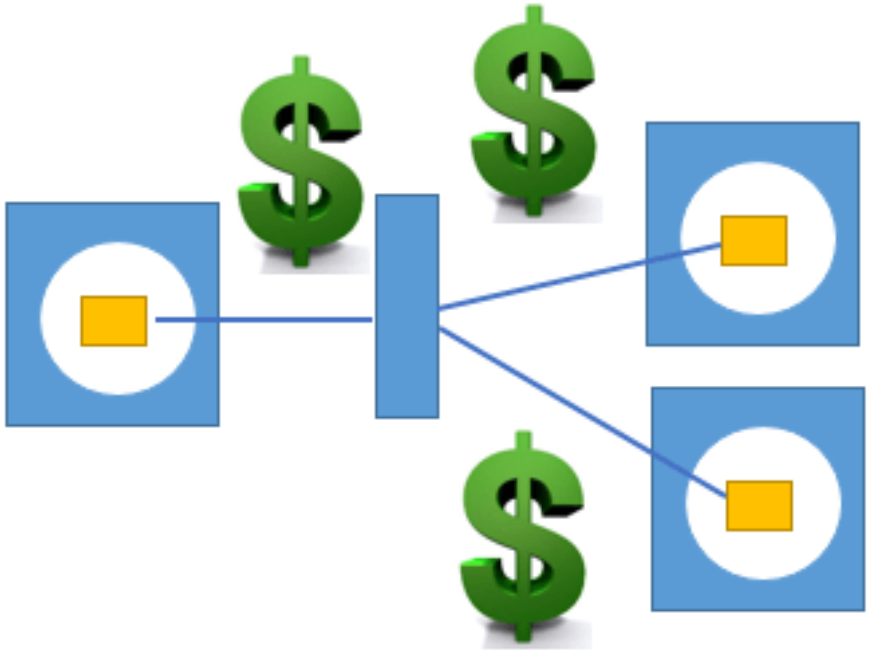
- En entornos modernos (nube híbrida, multi-nube, IoT), múltiples equipos, proveedores y organizaciones gestionan partes de la infraestructura.

Consecuencias prácticas si se cree verdadera

- Falta de coordinación en políticas de seguridad, monitoreo o configuración lleva a brechas operativas, conflictos de IP o problemas de cumplimiento normativo.

1.4 Falacias de las aplicaciones Distribuidas

EL COSTO DEL TRANSPORTE ES CERO



¿Por qué es una falacia?

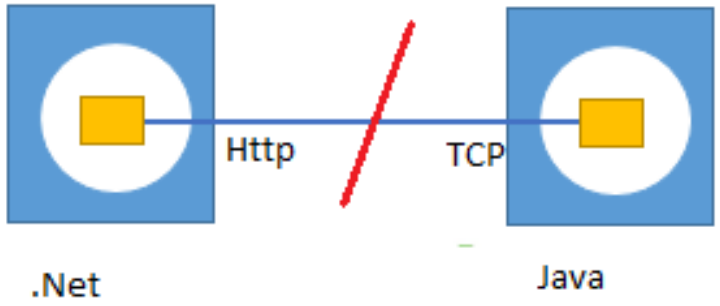
- Cada byte transmitido tiene un costo: monetario (en la nube o móviles), energético (en dispositivos IoT) y de rendimiento (procesamiento de protocolos).

Consecuencias prácticas si se cree verdadera

- Arquitecturas "chatty" (muchas llamadas pequeñas) o transferencias innecesarias elevan facturas de nube, agotan baterías y saturan CPUs.

1.4 Falacias de las aplicaciones Distribuidas

LA RED ES HOMOGÉNEA



¿Por qué es una falacia?

- Coexisten múltiples protocolos (HTTP, gRPC, MQTT), sistemas operativos, lenguajes, firewalls, NATs, proxies y dispositivos con capacidades muy distintas.

Consecuencias prácticas si se cree verdadera

- Suponer que todos los nodos se comunican igual causa fallos de interoperabilidad, especialmente en ecosistemas IoT o integraciones legacy.

Introducción a las Aplicaciones Distribuidas

1 Introducción

1.1 Conceptos de Aplicaciones Distribuidas

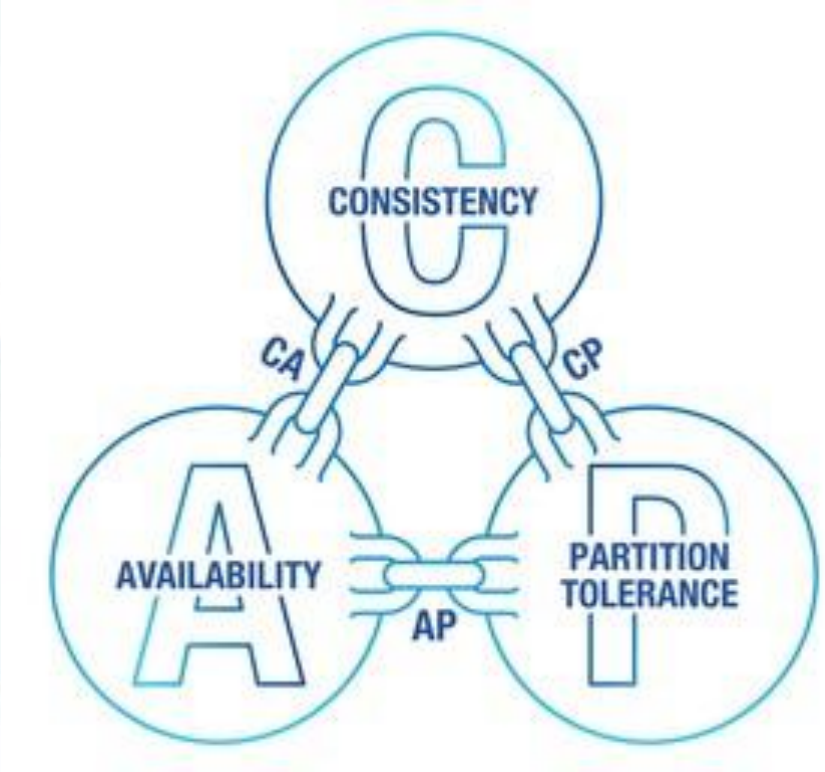
1.2 Evolución de las aplicaciones distribuidas

1.3 Principales desafíos y ventajas

1.4 Falacias de las aplicaciones Distribuidas

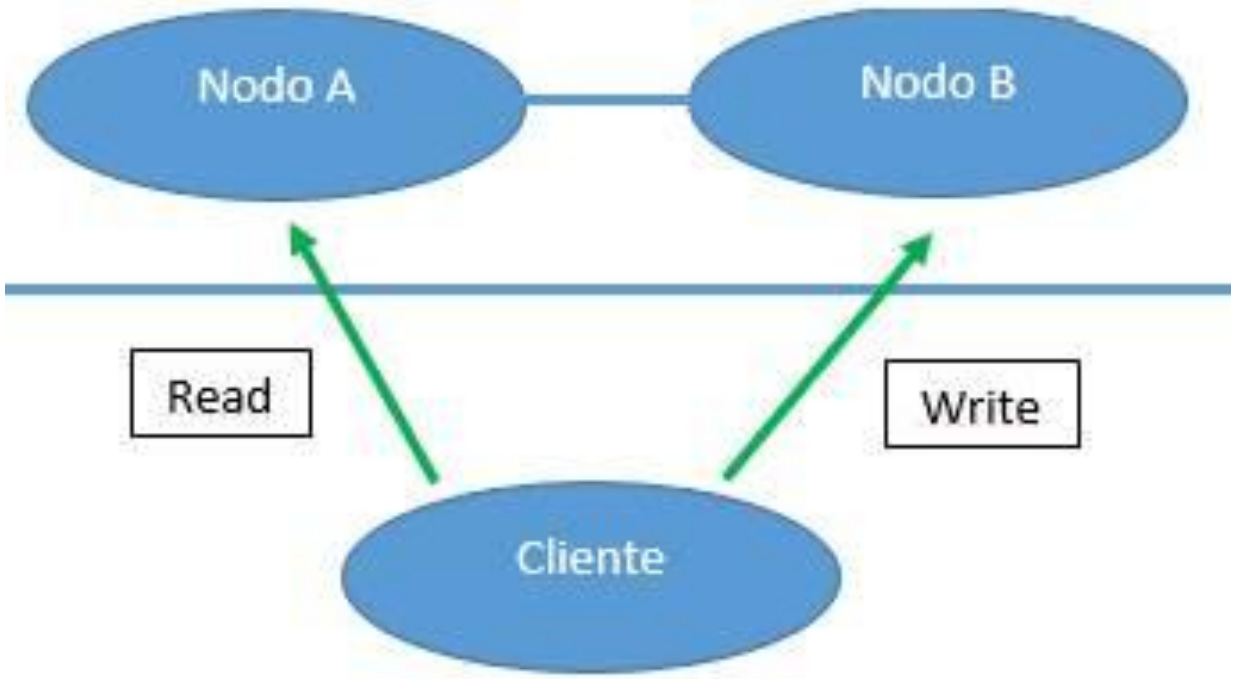
1.5 Teorema CAP

1.5 Teorema CAP



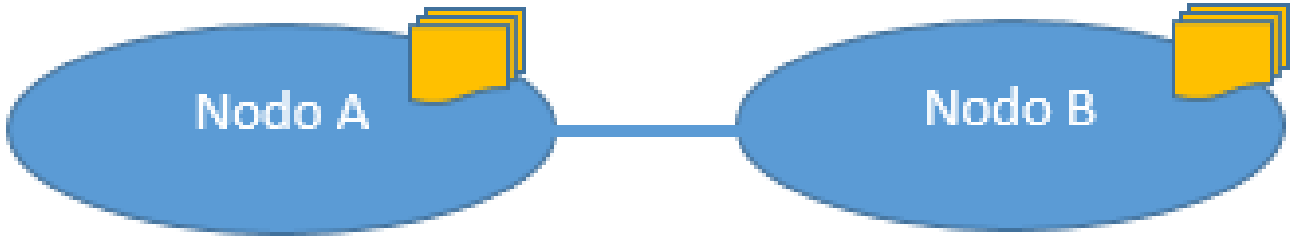
- El teorema CAP, propuesto por Eric Brewer en el año 2000, define el comportamiento de un sistema distribuido.
- Está formado por la primera letra de cada atributo: **C**onsistency, **A**vailability y **P**artition tolerance.
- Un sistema distribuido es un grupo de nodos conectados que comparten información entre sí.
- Un cliente puede escribir un dato interactuando con cualquier nodo y puede leerlo del mismo nodo o de uno diferente.

1.5 Teorema CAP



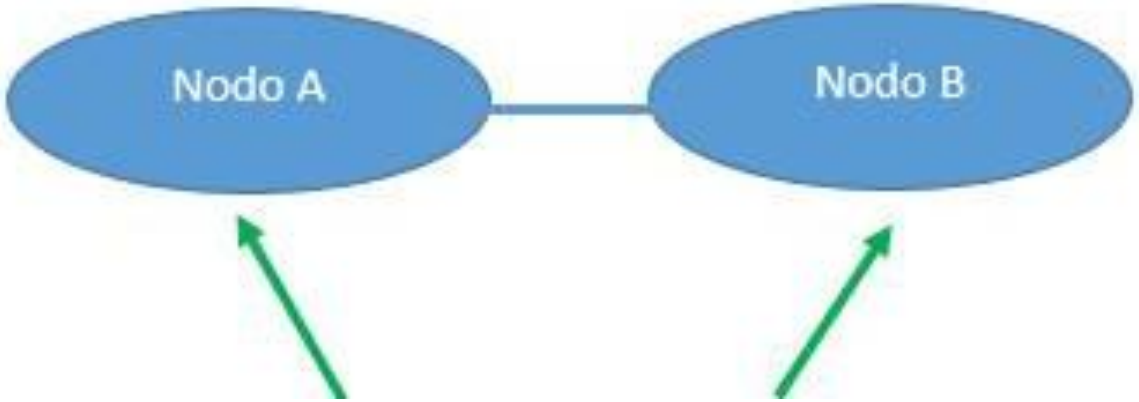
El teorema establece que, **para una operación de escritura seguida de una lectura, el sistema solo puede garantizar dos de tres atributos simultáneamente, debiendo sacrificar el tercero.**

1.5 Teorema CAP



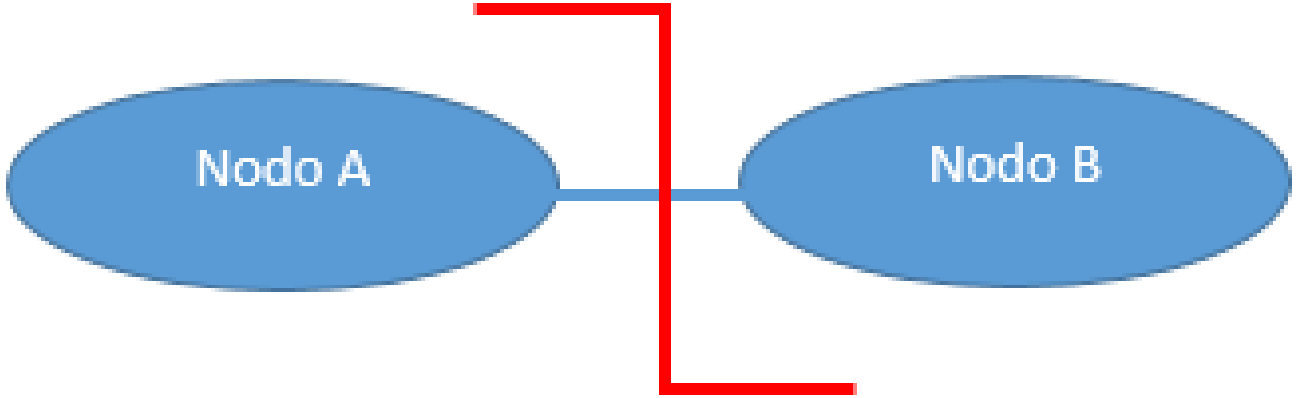
Consistency (Consistencia): Todos los nodos ven los mismos datos al mismo tiempo, garantiza que no se devuelva data desactualizada.

1.5 Teorema CAP



Availability (Disponibilidad): Si un nodo se encuentra abajo, el sistema tiene que responder a las peticiones enviadas.
Un nodo activo debe devolver una respuesta razonable en un periodo de tiempo razonable (sin errores o timeout).

1.5 Teorema CAP



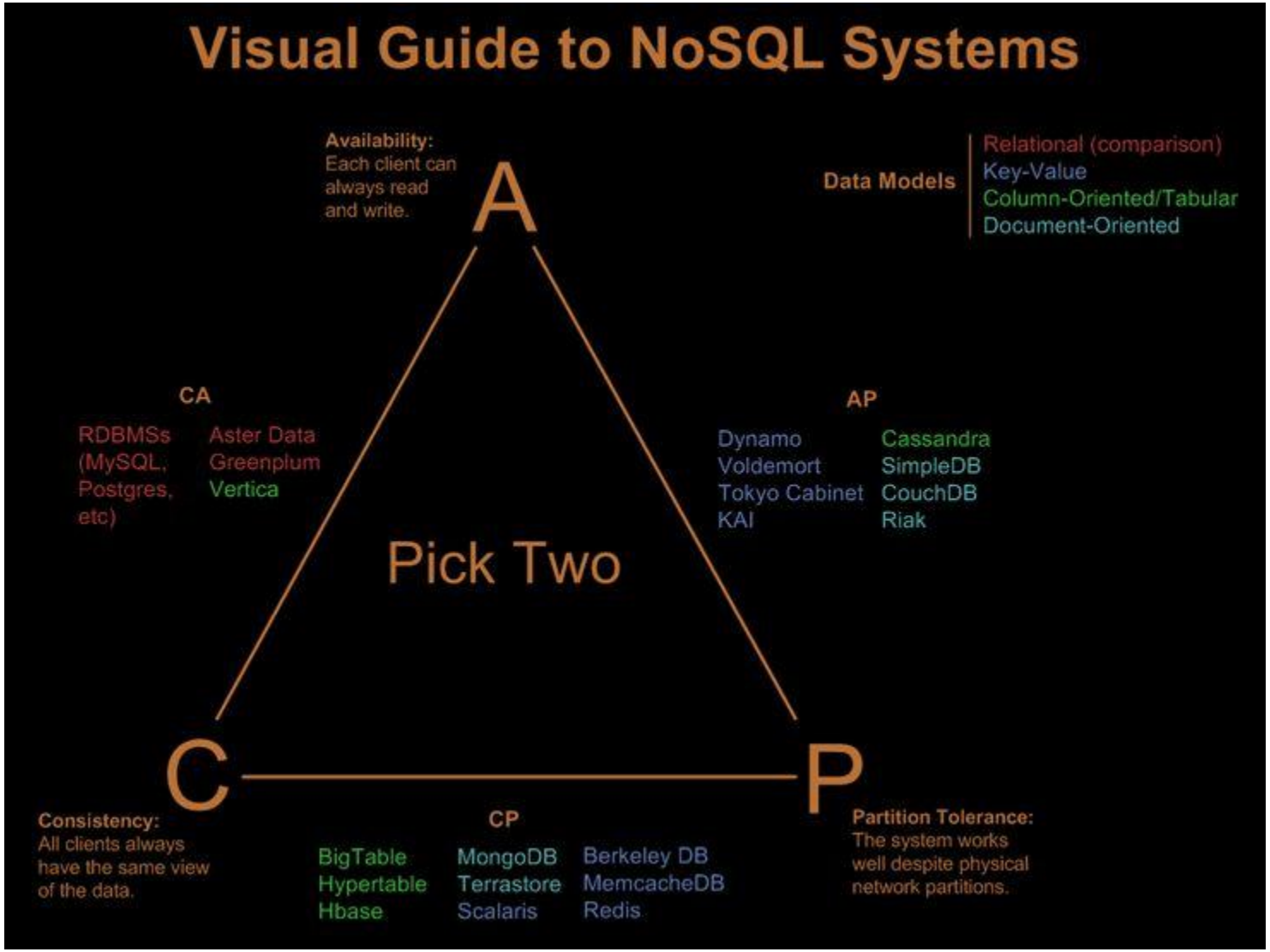
Partition tolerance (Tolerancia al particionado): Un sistema debe seguir funcionando aun cuando los nodos no se puedan comunicar entre sí (temporal o permanentemente).

1.5 Teorema CAP

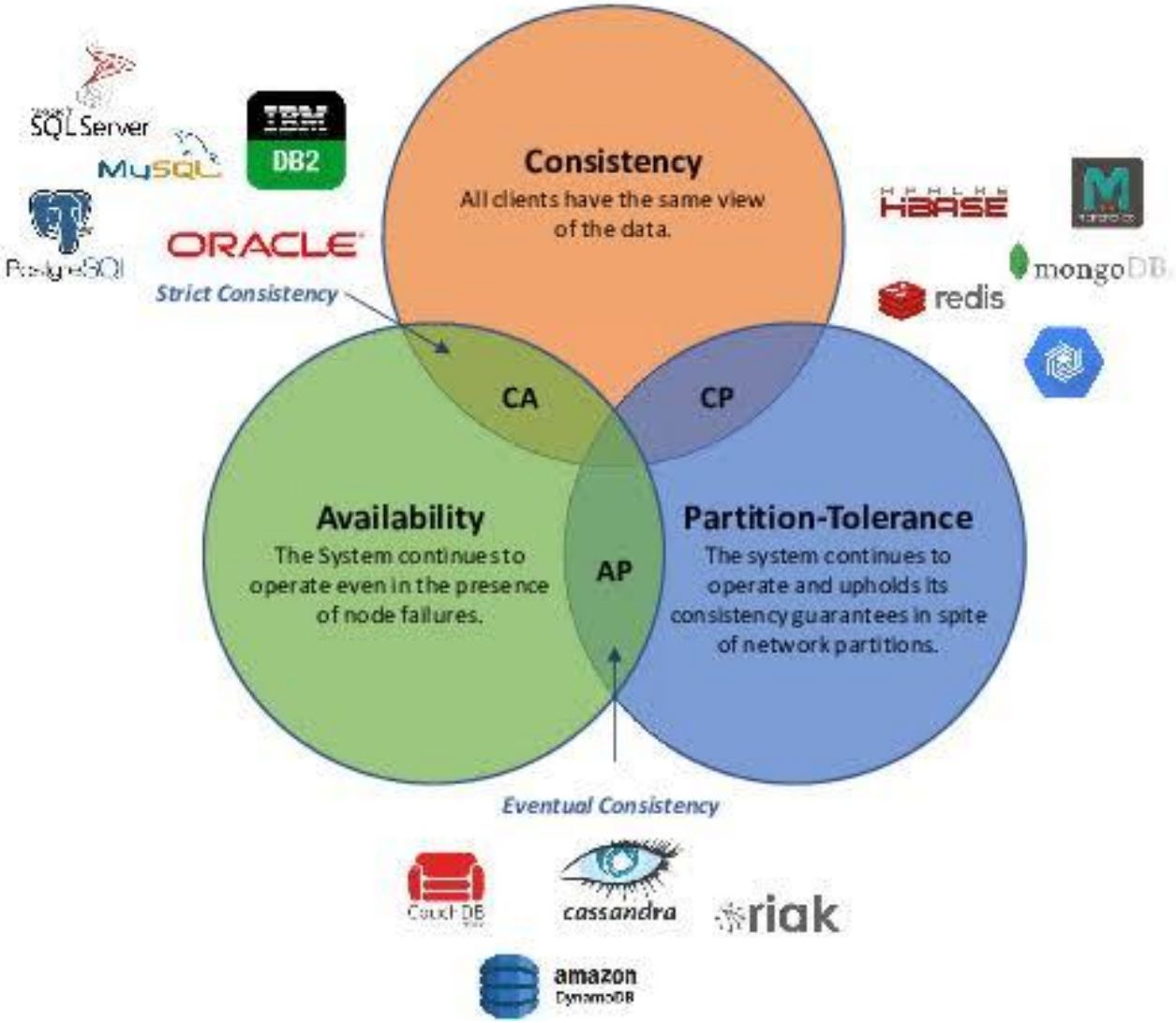
Teorema **CAP** solo podemos cumplir 2 de estos 3 atributos, así que **según el escenario debemos decidir que atributos necesitamos garantizar.**

Debemos tener en cuenta que en la computación distribuida la red nunca es confiable las caídas de red ocurren siempre y no las podemos controlar, por lo tanto, nos queda escoger entre mantener la consistencia o la accesibilidad

1.5 Teorema CAP



1.5 Teorema CAP



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Unidad I: FUNDAMENTOS DE APLICACIONES DISTRIBUIDAS

Gestión de concurrencia

Gestión de concurrencia

- Concurrencia en aplicaciones distribuidas
- Hilos de ejecución en la programación distribuida

Introducción

¿Qué es concurrencia?

Introducción

- La **concurrencia** permite manejar múltiples tareas simultáneamente en nodos remotos.
- Mejora el rendimiento y la escalabilidad en entornos como la nube o microservicios.
- Sin embargo, introduce desafíos únicos: **latencia de red, fallos parciales y problemas de consistencia de datos.**

Concurrencia en aplicaciones distribuidas

- **La concurrencia distribuida crea una ilusión de simultaneidad entre nodos que no ejecutan en paralelo real. Se basa en comunicación asíncrona, como HTTP o mensajería.**
- Los **problemas** comunes incluyen **condiciones de carrera**, donde dos nodos modifican el mismo recurso al mismo tiempo, generando inconsistencias.
- Otros riesgos: deadlocks, que bloquean procesos mutuamente, y starvation, donde tareas se ignoran indefinidamente.
- En bases de datos como Cassandra, se elige consistencia eventual sobre fuerte para priorizar disponibilidad.

Concurrencia en aplicaciones distribuidas

Paradigmas para Manejar Concurrencia

- **El modelo de Actores**, en frameworks como Akka, encapsula estado en actores que se comunican por mensajes inmutables, evitando compartición directa.
- **El modelo de Flujos Reactivos**, con RxJava, gestiona streams de datos asíncronos para procesar eventos en tiempo real.

Concurrencia en aplicaciones distribuidas

Paradigmas para Manejar Concurrencia

- **El modelo de Actores**, en frameworks como Akka, encapsula estado en actores que se comunican por mensajes inmutables, evitando compartición directa.
- **El modelo de Flujos Reactivos**, con RxJava, gestiona streams de datos asíncronos para procesar eventos en tiempo real.

Hilos de ejecución en la programación distribuida

¿Qué son los HILOS?

Hilos de ejecución en la programación distribuida

- **Los hilos son unidades básicas de concurrencia**, pero en distribuidos, pausan para esperar respuestas remotas, agregando overhead de red.
- En Java o Go, se crean con Thread o goroutines, integrados en pools para escalabilidad horizontal.
- Frameworks como Kubernetes orquestan contenedores con pools de hilos distribuidos.